P03 ��� Inspectable Typed Selectors and Selection Evidence

Executable research capsule with first-order unification

PBUI research prototype

2026-08-05

Contents

1 P03 - Inspectable Typed Selectors and Selection Evidence

1.1 Research question

Can PBUI selection predicates be made inspectable, serializable, optimizable, incrementally maintainable, and evidence-producing without eliminating the ability to call application-specific JavaScript?

This capsule implements a small typed selector language with:

- first-order term patterns;
- syntactic unification and most-general substitutions;
- a first-order predicate AST;
- explicit parameters, attributes, and pattern bindings;
- a reference evaluator;
- a simple indexed compiler;
- dependency and portability analysis;
- proof-relevant selection evidence;
- revision-sensitive evidence validation;
- an explicit opaque-lambda boundary.

1.2 Problem with callback-only selectors

A callback such as:

```
reference =>
  reference.type === "field" &&
  reference.value.docId === activeDocument &&
  reference.value.kind === "number"
```

is easy to write but hides information needed by a semantic runtime:

- dependencies;
- candidate indexes;
- serialization;
- worker execution;
- equivalence checking;
- explanation;
- monotonicity analysis;
- incremental invalidation;
- proof by structural induction.

P03 replaces the common first-order fragment with data while preserving a named opaque escape hatch.

1.3 Subject records

The selector evaluator consumes records:

```
interface SubjectRecord {
  sort: string;
  key: string;
```

```
  term: Term;
  attributes: Record<string, Scalar>;
}
```

Example:

```
{
  sort: "field",
  key: "doc-readings/temperature",
  term: field("doc-readings", "temperature", "number"),
  attributes: {
    docId: "doc-readings",
    name: "temperature",
    kind: "number",
    archived: false,
  },
}
```

The term supports structural pattern matching. Attributes support conventional indexed predicates. These representations deliberately overlap so the experiment can compare them.

1.4 Term language and unification

Terms are:

```
type Term =
  | { tag: "var"; name: string }
  | { tag: "atom"; value: Scalar }
  | { tag: "node"; symbol: string; args: Term[] };
```

A selector pattern can be:

```
field(?doc, ?name, "number")
```

When matched against:

```
field("doc-readings", "temperature", "number")
```

unification returns:

```
?doc  -> "doc-readings"
?name -> "temperature"
```

The substitution is retained in selection evidence and can be referenced by later predicates.

1.4.1 Algorithm

The implementation is a readable worklist transformation algorithm with these cases:

- delete equal terms;

- orient a variable to the left;
- eliminate a variable by substitution;
- decompose equal constructors;
- reject atom or constructor clashes;
- reject cyclic bindings through the occurs check.

The result is normalized to an idempotent substitution.

The prototype also provides:

- substitution application;
- composition;
- unifier checking;
- idempotence checking;
- a finite `factorSubstitution` experiment showing how a more specific solution factors through a computed general solution on selected variables.

1.5 Selector syntax

```
interface Selector {
  id: string;
  description?: string;
  from: string;
  pattern?: Term;
  where: Predicate;
}
```

Example:

```
const selector = {
  id: "active-numeric-field-in-document",
  from: "field",
  pattern: node(
    "field",
    variable("doc"),
    variable("name"),
    atom("number"),
  ),
  where: predicate.and(
    predicate.eq(
      expr.attribute("docId"),
      expr.parameter("document"),
    ),
    predicate.eq(
      expr.binding("doc"),
      expr.parameter("document"),
    ),
    predicate.eq(
      expr.attribute("archived"),
      expr.literal(false),
    ),
```

```
  ),
};
```

Predicate constructors currently include:

- literal true and false;
- equality and inequality;
- finite membership;
- conjunction;
- disjunction;
- negation;
- opaque predicates with declared dependencies.

Value expressions read:

- literals;
- record attributes;
- operation parameters;
- atom-valued unification bindings.

1.6 Selection evidence

Successful evaluation returns:

```
interface SelectionCandidate {
  record: SubjectRecord;
  substitution: Substitution;
  evidence: SelectionEvidence;
}
```

Evidence records:

- selector ID;
- store revision;
- selected subject coordinate;
- requested and encountered terms;
- the unifying substitution;
- a tree of predicate evidence.

A later commit calls `verifySelectionEvidence` against the current store revision and selector. This is not a cryptographic proof. It is a locally checkable certificate produced by the reference semantics.

1.7 Reference and compiled evaluators

1.7.1 Reference evaluator

`evaluateReference` scans every record in the requested sort and applies pattern matching and predicates. Its simplicity makes it the semantic oracle for tests.

1.7.2 Indexed compiler

`compileSelector` performs static analysis and extracts equality probes of the form:

```
attribute = literal
attribute = parameter
```

At execution time it chooses the smallest available indexed bucket, then applies the complete semantics to those candidates.

The compiler returns execution statistics:

```
{
  sourceCount,
  examined,
  accepted,
  usedIndex,
}
```

This is intentionally a small optimizer. It demonstrates why reification matters without pretending to be a general relational planner.

1.8 Dependency and portability analysis

`analyzeSelector` returns:

```
{
  sorts: ["field"],
  attributes: ["archived", "docId"],
  parameters: ["document"],
  opaquePredicates: [],
  portable: true,
}
```

A portable selector can be serialized as JSON. An opaque predicate cannot.

1.8.1 Opaque boundary

```
predicate.opaque(
  "bespoke-title-heuristic",
  ["attr:name", "param:minimumLength"],
  ({ record, parameters }) => /* arbitrary JavaScript */,
)
```

The declared dependencies permit conservative invalidation and documentation. They are trust assumptions, not inferred proofs. Opaque selectors are marked process-local and serialization fails explicitly.

1.9 Formal model

Let `R_S` be the finite set of records of sort `S`. A selector is interpreted relative to parameters `p` and store revision `r`:

$$[\![q]\!]_{R,p,r} \subseteq R_S \times \text{Substitution} \times \text{Evidence}\,.$$

For a record `x`:

1. if a pattern exists, compute `mgu(pattern, term(x))`;
2. evaluate the predicate under attributes, parameters, and that substitution;
3. construct evidence when both succeed.

Compiler correctness is the extensional statement:

$$\text{keys}(\text{execute}(\text{compile}(q), R, p)) = \text{keys}([\![q]\!]_{R,p}).$$

The tests compare both evaluators on fixtures and verify that the indexed evaluator examines fewer records for an indexed parameter.

1.10 Use of Baader-Snyder unification theory

The additional chapter is central to this capsule.

1.10.1 Most-general solutions

The selector pattern should not enumerate every ground way it can match. A unifier provides a substitution, and a most-general unifier captures all more specific solutions through further instantiation.

P03 uses this idea to make variable bindings reusable evidence rather than a one-time Boolean match.

1.10.2 Occurs check

The prototype rejects:

```
?x = f(?x)
```

because finite first-order substitutions cannot represent that equation without cyclic or infinite terms.

1.10.3 Syntactic scope

The chapter distinguishes syntactic unification from unification modulo an equational theory. P03 implements only the syntactic case. This is an explicit guarantee boundary.

For example, these terms do not unify in P03:

```
set(a, b)
set(b, a)
```

even if an application wishes to treat a set constructor as commutative. Such behavior would require named normalization or an AC unification module. The chapter explains why arbitrary equational unification cannot safely inherit the simple promise of one MGU.

1.10.4 MGU and coequalizer are not synonyms

The source gives a categorical reformulation in which a unifier equalizes a parallel pair and a most-general unifier has a factorization property. It also notes that a categorical coequalizer imposes uniqueness of the factor, which ordinary MGU definitions do not always supply without further restriction.

P03 therefore calls its result a substitution or unifier. It does not call it the quotient of the selector.

1.11 Tests and experiments

The thirteen executable tests cover:

1. MGU construction for a field pattern;
2. consistency of repeated variables;
3. occurs-check failure;
4. constructor clash;
5. substitution-composition law;
6. factorization of a specific unifier through a computed general one;
7. candidate production with unification and predicate evidence;
8. compiled/reference extensional equivalence;
9. indexed candidate reduction;
10. dependency extraction and serialization round trip;
11. opaque guarantee downgrade;
12. rejection of forged predicate evidence;
13. revision-sensitive evidence and rejection explanations.

Run:

```
npm test
npm run demo
```

1.12 Results established by the prototype

1.12.1 By construction

- The inspectable core is a finite AST.
- Portable selectors contain no executable callbacks.
- Pattern variables and substitutions are explicit data.
- The compiled evaluator always applies the full reference predicate after index narrowing.

1.12.2 By executable tests

- The unifier handles representative delete, orient, eliminate, decompose, clash, and occurs-check cases.
- Returned example substitutions are idempotent unifiers.
- A representative specific solution factors through the computed general solution.
- The indexed and reference evaluators return equal candidate keys on the fixture.
- Evidence fails validation after a store revision change.

1.12.3 Not yet proved

- soundness and completeness of the unifier for all first-order terms;
- principal/MGU status of every successful result;
- compiler correctness for all AST constructors;
- soundness of dependency extraction;
- optimizer correctness under future rewrites;
- security of evidence against forged JavaScript objects.

These are appropriate targets for P14-style mechanization.

1.13 Deliberate limitations

1. The store is in-memory and scalar-attribute only.
2. Pattern bindings can be consumed as scalar predicates only when they bind atoms.
3. The compiler uses one simple equality index rather than a cost-based join planner.
4. Predicate negation is local Boolean negation, not recursive logical negation.
5. Opaque dependency declarations are not verified.
6. Evidence is revision-sensitive and intentionally invalidated by any store mutation, even an unrelated one.
7. Equational, higher-order, sorted, and nominal unification are out of scope.

1.14 Integration seam

P01 should construct the subject coordinates used as record sort/key. P02 should expose committed occurrences as records. P04 can either consume P03 candidate results as extensional facts or compile a common relational fragment into both engines.

A future composed selection result should include:

```
{
  subjectIdentityEvidence,
  occurrenceActivationEvidence,
  selectorEvidence,
  ruleOrCapabilityEvidence,
}
```

The commit operation must revalidate each component at its own revision.

1.15 Research extensions

- prove unifier soundness and principality in Lean;
- add sort-aware and order-sorted terms;
- compare matching with full unification for selector patterns;
- implement a term-DAG/union-find optimizer and differential-test it;
- add join, projection, aggregation, and recursive query constructors;
- derive an incremental evaluator from change structures;
- introduce a checked monotone fragment;
- investigate AC/ACUI selectors for tags and capability sets;
- generate human-readable minimal rejection explanations;
- benchmark AST interpretation, bytecode, generated JavaScript, and relational plans.